

CPSC 304 Project Cover Page

Milestone #: 4

Date: Apr 2, 2026

Group Number: 80

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Boyi Zhang	57779811	f7r8h	zboyi0402@gmail.com
Runtan Zhang	51443323	t1y3u	zhangruntan@gmail.com
Xinghao Huang	81848509	u6u0n	xinghaohuang3@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above.

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

Project Milestone 4 - Project Implementation

Topic: Urban Bike Infrastructure Maintenance System

Project Summary

Our project is about urban bike infrastructure maintenance and operations. It concerns the real-world processes involved in operating and maintaining a city-wide shared bicycle system, including ensuring that bicycles deployed across urban stations remain safe, functional, and available for public use. This domain involves coordination between physical assets, maintenance staff, and operational policies to support reliable and efficient urban transportation.

What it Accomplished

Our project models an urban bike infrastructure maintenance system that manages shared bicycles deployed across multiple stations in a city. The system tracks bicycles, issues reports, maintenance tasks, and staff responsibilities.

Through the application interface, users can perform operations such as adding or updating bike records, reporting and managing issues, assigning maintenance tasks, and monitoring technician workloads. These operations demonstrate how a database can coordinate information about physical assets, operational activities, and personnel.

Overall, the project shows how a structured database system can support efficient maintenance management, issue tracking, and operational decision-making for a city-wide shared bike network.

Description of Final Schema

Component

Schema:

- Component(ComponentID: CHAR(4), StockQuantity: INTEGER, Material: VARCHAR(10), Price: FLOAT, Name: VARCHAR(20))

Description:

- Represents a component type in inventory, storing its identifying information and available stock quantity, which can be used by multiple bikes and required by multiple maintenance tasks.

Primary key:

- ComponentID

Candidate keys:

- Name

Foreign keys:

- None

Other Constraints:

- Name UNIQUE
- StockQuantity ≥ 0
- Price ≥ 0

ConsistOf

Schema:

- ConsistOf(ComponentID: CHAR(4), BikeID: CHAR(4))

Description:

- The ConsistOf relation represents the many-to-many relationship between Component and Bike. It records which components are part of a bike.

Primary key:

- (ComponentID, BikeID)

Candidate keys:

- None (other than primary key)

Foreign keys:

- **ComponentID**: Referencing the ComponentID attribute of the Component entity.
- **BikeID**: Referencing the BikeID attribute of the Bike entity.

Other Constraints:

- The total participation of Bike in ConsistOf cannot be enforced using standard foreign key or NOT NULL constraints. Therefore, this constraint would require an assertion to ensure that every Bike appears in at least one tuple of ConsistOf.

Need

Schema:

- Need(**ComponentID**: CHAR(4), **TaskID**: CHAR(4), **MaintenancelD**: CHAR(4))

Description:

- The Need relation represents the many-to-many relationship between Component and MaintenanceTask. It records which components are needed for a maintenance task.

Primary key:

- (ComponentID, TaskID, MaintenancelD)

Candidate keys:

- None (other than primary key)

Foreign keys:

- **ComponentID**: Referencing the ComponentID attribute of the Component entity.
- **(TaskID, MaintenancelD)**: referencing MaintenanceTask(TaskID, MaintenancelD).

Other Constraints:

- None

Bike

Schema:

- Bike(BikeID: CHAR(4), Brand: VARCHAR(10), LastServiceDate: DATE, DeploymentDate: DATE, Status: **VARCHAR(20)**, **StreetAddress**: VARCHAR(100), **PostalCode**: CHAR(6), **StationName**: VARCHAR(20))

Description:

- Represents an individual bike in the system, storing its identifying and operational information.
- The relationship, LocatedAt, is merged with the Bike entity because of a many-to-one relationship.

Primary key:

- BikeID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **StreetAddress & PostalCode & StationName**: a composite foreign key associating with the LocatedAt relationship and referencing the composite primary key (StreetAddress, PostalCode, StationName) of the Station entity.

Other Constraints:

- Status NOT NULL
- The ISA hierarchy between Bike and its subtypes (RegularBike and E-Bike) is disjoint and total. Therefore, each Bike instance must appear in exactly one subtype relation, and the subtype relations are mutually exclusive.

Station

Schema:

- Station(StreetAddress: VARCHAR(100), PostalCode: CHAR(6), StationName: VARCHAR(20), NumberOfBike: INTEGER, InstallationDate: DATE, TotalCapacity: INTEGER)

Description:

- Represents a bike station in the system, and stores location information and capacity details.

Primary key:

- (StreetAddress, PostalCode)

Candidate keys:

- None (other than primary key)

Foreign keys:

- None

Other Constraints:

- NumberOfBike ≥ 0
- TotalCapacity ≥ 0
- StationName NOT NULL

RegularBike

Schema:

- RegularBike(**BikeID**: CHAR(4))

Description:

- RegularBike is a subtype of Bike under a disjoint and total specialization. It represents standard bikes that do not contain additional subtype-specific attributes.

Primary key:

- BikeID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **BikeID**: associating with the IsA relationship and referencing the BikeID attribute of the Bike entity

Other Constraints:

- None

EBike

Schema:

- E Bike(**BikeID**: CHAR(4), BatteryLevel: INTEGER)

Description:

- E Bike is a subtype of Bike under a disjoint and total specialization. It extends Bike by adding the BatteryLevel attribute, which tracks the charge level of electric bikes.

Primary key:

- BikeID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **BikeID**: associating with the IsA relationship and referencing the BikeID attribute of the Bike entity

Other Constraints:

- $0 \leq \text{BatteryLevel} \leq 100$

Customer

Schema:

- Customer(CustomerID: CHAR(4), Name: VARCHAR(30), Address: VARCHAR(100), Email: VARCHAR(30), Phone#: VARCHAR(20), RegisteredDate: DATE)

Description:

- Represents registered users of the system. It stores personal and contact information, including registration details. CustomerID uniquely identifies each customer.

Primary key:

- CustomerID

Candidate keys:

- None

Foreign keys:

- None

Other Constraints:

- Email UNIQUE NOT NULL
- RegisteredDate NOT NULL

Report

Schema:

- Report(CustomerID: CHAR(4), IssueID: CHAR(4))

Description:

- The Report relation represents the many-to-many relationship between Customer and Issue. It records which customer reports which issue.

Primary key:

- (CustomerID, IssueID)

Candidate keys:

- None (other than primary key)

Foreign keys:

- **CustomerID**: referencing the CustomerID attribute of the Customer entity
- **IssueID**: referencing the IssueID of the Issue entity

Other Constraints:

- None

Issue

Schema:

- Issue(IssueID: CHAR(4), ReportTime: TIMESTAMP, Result: VARCHAR(20), ConditionScore: FLOAT, **BikeID**: CHAR(4), **MaintenanceID**: CHAR(4), Description: CLOB, **InspectorID**: CHAR(4))

Description:

- Represents reported problems associated with bikes.
- The relationships, Experience, Check, and Fix, are merged with the Issue entity because of many-to-one relationships.

Primary key:

- IssueID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **BikeID**: associating with the Experience relationship, referencing the BikeID attribute of the Bike entity.
- **MaintenanceID**: associating with the Fix relationship, referencing the MaintenanceID attribute of the Maintenance Entity.
- **InspectorID**: associating with the Check relationship, referencing the StaffID attribute of the Inspector entity (note that Inspector is a subtype of staff).

Other Constraints:

- Result NOT NULL
- Description NOT NULL
- ConditionScore NOT NULL
- BikeID NOT NULL
- $0.0 \leq \text{ConditionScore} \leq 5.0$

MaintenanceTask

Schema:

- MaintenanceTask(TaskID: CHAR(4), **MaintenanceID**: CHAR(4), Duration: TIME, StartTime: **TIMESTAMP DEFAULT CURRENT_TIMESTAMP**, EndTime: TIMESTAMP, **TechnicianID**: CHAR(4))

Description:

- MaintenanceTask is a weak entity that represents individual tasks within a maintenance record.
- The relationships, PartOf and Perform, are merged with the MaintenanceTask entity because of both many-to-one relationship and weak entity relationship.

Primary key:

- TaskID & MaintenanceID (Note that, TaskID by itself is a partial key of the weak entity MaintenanceTask)

Candidate keys:

- None (other than primary key)

Foreign keys:

- **MaintenanceID**: associating with the PartOf relationship, referencing the MaintenanceID attribute of the Maintenance entity.
- **TechnicianID**: associating with the Perform relationship, referencing the StaffID attribute of the Technician entity (note that Technician is a subtype of Staff).

Other Constraints:

- MaintenanceID NOT NULL

Maintenance

Schema:

- Maintenance(MaintenanceID: CHAR(4), PriorityLevel: INTEGER, CompletionStatus: VARCHAR(10), RepairQuotation: CLOB)

Description:

- The Maintenance entity represents a maintenance record associated with one or more issues. It stores priority level, completion status, and repair quotation information. Each maintenance record may consist of multiple maintenance tasks.

Primary key:

- MaintenanceID

Candidate keys:

- None (other than primary key)

Foreign keys:

- None

Other Constraints:

- PriorityLevel ≥ 0
- CompletionStatus NOT NULL

Staff

Schema:

- Staff(StaffID: CHAR(4), Name: VARCHAR(30), Contact: VARCHAR(20), AssignedArea: CLOB, RoleID: CHAR(4), **ManagerID: CHAR(4)**)

Description:

- The Staff entity represents employees in the system.
- The relationship, Supervise, is merged with the Staff entity because of a many-to-one relationship. And since Manager is a subtype of Staff, an additional foreign key, ManagerID, is added in order to reference a manager.

Primary key:

- StaffID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **ManagerID**: associating with the Supervise relationship, referencing the StaffID attribute of the Manager entity (note that Manager is a subtype of Staff, and a manager will supervise different types of staff).

Other Constraints:

- The ISA hierarchy between Staff and its subtypes (Technician, Inspector, Manager) is disjoint and total. Therefore, each Staff instance must appear in exactly one subtype relation, and the subtype relations are mutually exclusive.

Technician

Schema:

- Technician(**StaffID**: CHAR(4))

Description:

- Technician is a subtype of Staff under a disjoint and total specialization. It represents staff members responsible for performing maintenance tasks.

Primary key:

- StaffID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **StaffID**: associating with the IsA relationship, referencing the StaffID attribute of the Staff entity.

Other Constraints:

- None

Inspector

Schema:

- Inspector(**StaffID**: CHAR(4))

Description:

- Inspector is a subtype of Staff under a disjoint and total specialization. It represents staff members responsible for inspecting and evaluating reported issues.

Primary key:

- StaffID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **StaffID**: associating with the IsA relationship, referencing the StaffID attribute of the Staff entity.

Other Constraints:

- None

Manager

Schema:

- Manager(**StaffID**: CHAR(4))

Description:

- Manager is a subtype of Staff under a disjoint and total specialization. It represents staff members responsible for supervising other staff members.

Primary key:

- StaffID

Candidate keys:

- None (other than primary key)

Foreign keys:

- **StaffID**: associating with the IsA relationship, referencing the StaffID attribute of the Staff entity.

Other Constraints:

- None

Summary of Schema Changes

The final schema is largely consistent with the original schema we submitted, but includes one major change and a few minor adjustments based on feedback .

The main change concerns the **Staff** relation. In the original schema, **ManagerID** was defined with a **NOT NULL constraint**, which forced every staff member to have a manager. However, this assumption is not valid in practice because managers are also staff members, and top-level managers may not have a supervisor. To address this issue, we **removed the NOT NULL constraint on ManagerID**, allowing it to take **NULL values**. This ensures that the schema can correctly represent managers without supervisors while still maintaining the foreign key relationship when a manager exists.

In addition to this major change, we made minor improvements to better reflect realistic data requirements. For example, the **Status** attribute in the **Bike** relation was expanded from **VARCHAR(10)** to **VARCHAR(20)** to accommodate longer and more descriptive status values (such as “Out of Service” or “In Maintenance”).

Other than that, the domain of **StartTime** attribute of the **MaintenanceTask** relation is changed from “**TIMESTAMP**” to “**TIMESTAMP DEFAULT CURRENT_TIMESTAMP**” for better demonstration in future milestone (i.e., Milestone 5)

Overall, these changes improve the **realism, flexibility, and correctness** of the schema while preserving its original structure and design.

Visual Representation of The State of Database

BikeStation :

	STREETADDRESS	POSTALCODE	STATIONNAME
1	123 Main St	V6T1Z4	UBC Station
2	456 Oak St	V6T1Z5	Downtown Station
3	789 Pine St	V6T1Z6	CityHall Station
4	321 Cedar St	V6T1Z7	Kits Station
5	654 Maple St	V6T1Z8	Granville Station

Bike :

	BIKEID	BRAND	LASTSERVICEDATE	DEPLOYMENTDATE	STATUS	STREETADDRESS	POSTALCODE
1	B001	Trek	01/01/25	10/01/25	Available	123 Main St	V6T1Z4
2	B002	Giant	02/01/25	11/01/25	Maintenance	123 Main St	V6T1Z4
3	B003	Norco	03/01/25	12/01/25	Available	456 Oak St	V6T1Z5
4	B004	Cube	04/01/25	13/01/25	Out of Service	456 Oak St	V6T1Z5
5	B005	Special	05/01/25	14/01/25	In Use	789 Pine St	V6T1Z6
6	B006	Trek	06/01/25	15/01/25	Maintenance	321 Cedar St	V6T1Z7
7	B007	Giant	07/01/25	16/01/25	In Use	321 Cedar St	V6T1Z7
8	B008	Norco	08/01/25	17/01/25	Available	654 Maple St	V6T1Z8
9	B009	Cube	09/01/25	18/01/25	In Use	654 Maple St	V6T1Z8
10	B010	Special	10/01/25	19/01/25	Out of Service	789 Pine St	V6T1Z6

Component :

	COMPONENTID	STOCKQUANTITY	MATERIAL	PRICE	NAME
1	C001	50	Steel	25	Brake
2	C002	40	Aluminum	35	Wheel
3	C003	30	Rubber	15	Tire
4	C004	20	Plastic	12	Seat
5	C005	25	Steel	18	Chain

ConsistOf :

	COMPONENTID	BIKEID
1	C001	B001
2	C002	B002
3	C003	B003
4	C004	B004
5	C005	B005

CustomerAccount :

	CUSTOMERID	EMAIL	REGISTEREDDATE
1	CU01	alice@email.com	01/01/25
2	CU02	bob@email.com	02/01/25
3	CU03	carol@email.com	03/01/25
4	CU04	david@email.com	04/01/25
5	CU05	emma@email.com	05/01/25

EBike:

	BIKEID	BATTERYLEVEL
1	B006	80
2	B007	70
3	B008	60
4	B009	90
5	B010	75

Staff (Inspector):

	STAFFID
1	S006
2	S007
3	S008
4	S009
5	S010

IssueRecord:

	ISSUEID	REPORTTIME	CONDITIONSCORE	DESCRIPTION	BIKEID	MAINTENANCEID	INSPECTORID
1	I001	01/01/26 09:00:00.000	2	Loose brake	B001	M001	S006
2	I002	02/01/26 09:00:00.000	3	Flat tire	B002	M002	S007
3	I003	03/01/26 09:00:00.000	4	Broken chain	B003	M003	S008
4	I004	04/01/26 09:00:00.000	1	Minor scratch	B004	M004	S009
5	I005	05/01/26 09:00:00.000	5	Frame damage	B005	M005	S010
6	I006	29/03/26 13:16:13.424	2	Loose brake	B001	M001	S006

IssueRule:

	CONDITIONSCORE	DESCRIPTION	RESULT
1	1	Minor scratch	Ignore
2	2	Loose brake	Repair
3	3	Flat tire	Repair
4	4	Broken chain	Urgent
5	5	Frame damage	Replace

Maintenance:

	MAINTENANCEID	PRIORITYLEVEL	COMPLETIONSTATUS	REPAIRQUOTATION
1	M001	1	Pending	Brake repair
2	M002	2	Complete	Wheel replacement
3	M003	1	Pending	Tire repair
4	M004	3	Pending	Seat replacement
5	M005	2	Complete	General inspection

MaintenanceTask:

	TASKID	MAINTENANCEID	DURATION	STARTTIME	ENDTIME	TECHNICIANID
1	T001	M001	+00 01:00:00.000000	01/01/26 10:00:00.000	01/01/26 11:00:00.000	S001
2	T002	M002	+00 02:00:00.000000	02/01/26 10:00:00.000	02/01/26 12:00:00.000	S001
3	T003	M003	+00 01:00:00.000000	03/01/26 09:00:00.000	03/01/26 10:00:00.000	S001
4	T004	M004	+00 03:00:00.000000	04/01/26 08:00:00.000	04/01/26 11:00:00.000	S001
5	T005	M005	+00 01:00:00.000000	05/01/26 13:00:00.000	05/01/26 14:00:00.000	S001

Staff (Manager):

	STAFFID
1	S011
2	S012
3	S013
4	S014
5	S015

Need:

	COMPONENTID	TASKID	MAINTENANCEID
1	C001	T001	M001
2	C002	T002	M002
3	C003	T003	M003
4	C004	T004	M004
5	C005	T005	M005

RegularBike:

	BIKEID
1	B001
2	B002
3	B003
4	B004
5	B005

Report:

	CUSTOMERID	ISSUEID
1	CU01	I001
2	CU02	I002
3	CU03	I003
4	CU04	I004
5	CU05	I005

Staff:

	STAFFID	NAME	CONTACT	ASSIGNEDAREA	ROLEID	MANAGERID
1	S011	Alice	111	UBC	R1	(null)
2	S012	Bob	222	Downtown	R1	(null)
3	S013	Carol	333	CityHall	R1	(null)
4	S014	David	444	Kits	R1	(null)
5	S015	Emma	555	Granville	R1	(null)
6	S001	Tom	tom@technician.com	UBC	R2	S011
7	S002	Jerry	jerry@technician.com	UBC	R2	S011
8	S003	Mark	mark@technician.com	Downtown	R2	S012
9	S004	Luke	luke@technician.com	CityHall	R2	S013
10	S005	John	john@technician.com	Kits	R2	S014
11	S006	Anna	666	UBC	R3	S011
12	S007	Bella	777	Downtown	R3	S012
13	S008	Cindy	888	CityHall	R3	S013
14	S009	Diana	999	Kits	R3	S014
15	S010	Eva	101	Granville	R3	S015

StationAddress:

	STREETADDRESS	POSTALCODE	STATIONNAME
1	123 Main St	V6T1Z4	UBC Station
2	456 Oak St	V6T1Z5	Downtown Station
3	789 Pine St	V6T1Z6	CityHall Station
4	321 Cedar St	V6T1Z7	Kits Station
5	654 Maple St	V6T1Z8	Granville Station

StationInfo:

	STATIONNAME	NUMBEROFBIKE	INSTALLATIONDATE	TOTALCAPACITY
1	UBC Station	20	01/01/24	40
2	Downtown Station	25	01/02/24	50
3	CityHall Station	18	01/03/24	35
4	Kits Station	15	01/04/24	30
5	Granville Station	22	01/05/24	45

Technician:

	STAFFID
1	S001
2	S002
3	S003
4	S004
5	S005

Non-Hardcoded SQL Queries

Insert

SQL Statement:

- `INSERT INTO MaintenanceTask (TaskID, MaintenanceID, TechnicianID) VALUES (:TaskID, :MaintenanceID, :TechnicianID)`

File name:

- `appService.js`

Line number(s):

- 676-677

Update

SQL Statement:

- `SELECT Status
FROM Bike
WHERE TRIM(UPPER(BikeID)) = TRIM(UPPER(:bikeID))`

File name:

- `appService.js`

Line number(s):

- 347-349

Delete

SQL Statement:

- `DELETE FROM IssueRecord WHERE IssueID = :issueId`

File name:

- `appService.js`

Line number(s):

- 490

Selection

SQL Statement:

- `SELECT BikeID, Brand, LastServiceDate, DeploymentDate, Status,
StreetAddress, PostalCode
FROM Bike
WHERE 1=1`

File name:

- `appService.js`

Line number(s):

- 417-419

Projection

SQL Statement:

- ```
SELECT IR.IssueID, IR.ReportTime, IR.ConditionScore,
 IR.Description, IR.BikeID, IR.MaintenanceID, IR.InspectorID,
 IRL.Result
FROM IssueRecord IR
 JOIN IssueRule IRL ON IR.ConditionScore = IRL.ConditionScore
 AND IR.Description = IRL.Description
```

File name:

- `appService.js`

Line number(s):

- 540-544

## Join

SQL Statement:

- ```
SELECT *  
FROM Technician T  
WHERE T.StaffID = :TechnicianID
```

File name:

- `appService.js`

Line number(s):

- 699-701

Hardcoded SQL Queries

Aggregation with GROUP BY

SQL Statement:

- ```
SELECT StreetAddress, PostalCode, COUNT(*) AS BikeCount
FROM Bike
GROUP BY StreetAddress, PostalCode
ORDER BY PostalCode, StreetAddress
```

File name:

- `appService.js`

Line number(s):

- 461-464

Description:

- This query counts the number of bikes at each station by grouping bikes based on station address and postal code.

## Aggregation with HAVING

SQL Statement:

- ```
SELECT BikeID, COUNT(*) AS IssueCount
FROM IssueRecord
GROUP BY BikeID
HAVING COUNT(*) >= 1
```

File name:

- `appService.js`

Line number(s):

- 561-564

Description:

- This query returns bikes that have recorded issues, filtering grouped results using a HAVING condition.

Nested Aggregation with GROUP BY

SQL Statement:

- ```
SELECT S.StaffID, S.Name, COUNT(*) AS NumberOfTask
FROM Staff S, Technician T, MaintenanceTask MT
WHERE S.StaffID = T.StaffID AND MT.TechnicianID = T.StaffID
GROUP BY S.StaffID, S.Name
HAVING COUNT(MT.TaskID) >
 (SELECT AVG(NumberOfTask)
 FROM
 (SELECT COUNT(*) As NumberOfTask
 FROM MaintenanceTask
 GROUP BY TechnicianID
)
)
```



)

File name:

- `appService.js`

Line number(s):

- 740-751

Description:

- This query finds technicians whose workload exceeds the average workload, using a nested aggregation subquery inside the HAVING clause.

## Division

SQL Statement:

- ```
SELECT S.StaffID, S.name
FROM Staff S, Technician T
WHERE S.StaffID = T.StaffID
      AND NOT EXISTS (
        (SELECT MT.TaskID
         FROM MaintenanceTask MT)
        MINUS
        (SELECT MT2.TaskID
         FROM MaintenanceTask MT2
         WHERE MT2.TechnicianID = T.StaffID)
      )
```

File name:

- `appService.js`

Line number(s):

- 767-777

Description:

- This query finds technicians who have worked on every maintenance task by using a division operation implemented with NOT EXISTS and MINUS.